BSc (Hons) in **Computer Science**

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: The standard used to determine how successfully the agent completes its goal.

E – Environment: The surroundings and conditions in which the agent operates and makes decisions.

A – Actuators: The tools or mechanisms the agent uses to perform actions and influence the environment.

S – Sensors: The methods or devices the agent uses to collect information from the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The physical environment state is represented by self.width, self.height, self.agent_pos, self.walls, self.food_positions, self.opponents, and self.collision. These variables describe the grid size, the locations of objects and agents, and whether a collision has occurred.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Classification: Multi-Agent.

Justification: The environment is Multi-Agent because self.opponents stores multiple opponent agents that operate alongside the main agent. Their movements and interactions, such as collisions that reduce the agent's score, directly affect the agent's performance and decision making.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all percepts an agent has received from the moment it starts operating until the present time.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The get_percept() method represents the Sensors component of PEAS because it gathers environmental information and provides it to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Classification: Partially Observable.
The get_percept() method gives the agent some useful information, including its own position, all opponent positions, current score, remaining food count, collision status, and whether food or a toxic trap exists at its current location. However, it does not reveal the exact locations of all food items, walls, or toxic traps. Therefore, the agent cannot observe the complete environment state and must act using incomplete information.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Deducting points for hitting a wall updates the Performance Measure because self.score is the metric used to evaluate how well the agent performs.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

A performance measure should evaluate the results of the agent's actions in the environment rather than its internal processing. This ensures the agent is rewarded for achieving its goals, such as avoiding walls and collecting food, instead of simply performing more computations. Measuring external outcomes provides an objective way to assess how successfully the agent completes its task.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment remains Partially Observable. Although self.toxic_traps is added to the environment and can affect the agent's performance, hiding this information from the agent's sensors means the agent cannot directly perceive the trap locations. As a result, the agent must make decisions using incomplete information, so the environment is classified as Partially Observable.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new

boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the smells_toxin sensor improves the agent's rationality by giving it an additional percept about danger in the environment. This information allows the agent to make better-informed decisions instead of acting blindly. By using the toxin percept when selecting future actions, the agent can avoid harmful states and choose actions expected to produce a higher overall utility.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic "Vacuum World Exploit" involved a robot vacuum that was rewarded with +1 point every time it collected dirt. A rational agent exploited this faulty metric by cleaning a patch of dirt, emptying its dust container back onto the floor, and repeatedly cleaning the same spot to earn unlimited points. This demonstrated that a poorly designed performance measure can encourage unwanted behavior. The correct metric should reward the environment remaining clean over time rather than simply counting cleaning actions.

Finalize and Lock Lab 01 Analytical Evaluation

FACULTY OF COMPUTING